TABLE OF CONTENTS

TABLE OF CONTENTS 1
Republic of the Philippines 2
Western Mindanao State University 2
College of Computing Studies — Zamboanga City 2
Capstone 2 Project Update / Progress Report 2
Project Title 2
Project Description 2
Problem Background 2
System Purpose and Vision 3
Core Operational User Access 3
Theoretical and Engineering Foundations 3
AI/ML Models, Algorithms, IoT, and Other Key Technologies Implemented 4
A. Smart Pillow and IoT Subsystem 4
B. Mobile Application and Computational Algorithms 5
C. Technology Stack and Operational Responsibilities 8
Current Completion Percentage 9
Provisional Overall Project Completion: 82% 9
Verification Performed on October 2, 2026 11
Project Status 11
A. Features and Functions Already Working 11
B. Features Currently Under Development or Requiring Final Validation 12
C. Features Not Yet Implemented or Reserved for Future Development 13
Data and Dataset Details 13
A. Current Data Sources 13
B. Types of Data 13

C. Data Scope and Limitations 14
Screenshot and Evidence Plan 14
Immediate Next Milestones 15
Summary 16

Republic of the Philippines

Western Mindanao State University

College of Computing Studies — Zamboanga City

Capstone 2 Project Update / Progress Report

Reporting date: October 2, 2026

Group members: [Insert complete names, program, and section]

Capstone research adviser: [Insert adviser's complete name and title]

Project Title

HAGOKILLER: An IoT-Enabled Smart Pillow for Snoring Monitoring, Automated Positional Intervention, and Privacy-Preserving Wellness Guidance

Official system acronym: HAGOKILLER

Project Description

Problem Background

Snoring is commonly treated as a minor inconvenience, yet persistent or intense snoring may disrupt sleep and may indicate that a person should seek professional evaluation. Conventional pillows cannot observe snoring patterns, document when an episode occurs, or respond when repeated snoring is detected. Mobile applications that rely only on a phone microphone may also be separated from the physical intervention mechanism and may require users to interpret raw recordings without sufficient context.

HAGOKILLER addresses this limitation through a closed-loop smart-pillow ecosystem. An ESP32-based pillow monitors acoustic activity, applies configurable detection conditions, and can activate an air pump to adjust the pillow when sustained possible snoring

is observed. A companion mobile application receives events through Bluetooth Low Energy (BLE), preserves them locally, computes daily and monthly summaries, and translates the measurements into understandable wellness information.

The system is intended for wellness monitoring and behavior reflection. It does not diagnose sleep apnea or any other medical condition, and it does not replace assessment or treatment by a licensed healthcare professional.

System Purpose and Vision

HAGOKILLER is designed to unite sensing, physical intervention, longitudinal monitoring, and safety-bounded guidance in one privacy-conscious platform. Its principal purpose is to help users recognize recurring snoring patterns and understand how often the pillow intervenes without requiring continuous internet connectivity.

The smart pillow performs local monitoring and intervention, while the React Native mobile application provides device pairing, event synchronization, historical logs, graphical analytics, daily check-ins, wellness-plan generation, and device configuration. Deterministic rules remain the primary safety mechanism. For normal and elevated readings, an optional on-device language model can personalize low-risk actions, while critical results bypass generative AI and retain fixed clinician-directed guidance.

Core Operational User Access

Smart-pillow user: Creates a local profile, pairs the pillow through a seven-digit PIN, reviews sleep information, completes an optional daily activity check-in, and receives wellness guidance.

Device owner or operator: Configures the snoring-event threshold, rolling detection window, microphone shift, and pump duration through the mobile application.

Prototype evaluator or researcher: Uses mock BLE behavior and seeded events when native hardware is unavailable, allowing repeatable demonstrations and interface evaluation.

Theoretical and Engineering Foundations

Internet of Things and closed-loop control: The microphone, ESP32, pump, valve, and mobile client form a sensing–decision–intervention–feedback cycle.

Signal-threshold event detection: Acoustic intensity values are evaluated against configurable conditions and a rolling observation window before an intervention is triggered.

Local-first computing: User profiles, device settings, raw events, check-ins, plans, and feedback are stored locally so that core functions remain available without continuous internet access.

Human-centered data interpretation: Raw event measurements are converted into severity classifications, temporal trends, charts, explanations, and manageable wellness actions.

Safety-bounded decision support: Deterministic rules control critical guidance and constrain any optional generative output.

Software quality and testability: Typed interfaces, modular services, validation, schema migration, error handling, and automated tests support maintainability and reliability.

AI/ML Models, Algorithms, IoT, and Other Key Technologies Implemented

A. Smart Pillow and IoT Subsystem

ESP32-based embedded controller

The ESP32 is the intended central controller of the smart pillow. It receives microphone measurements, evaluates detection settings, controls the intervention hardware, records event information, and exposes a BLE interface for the mobile application.

Microphone-based acoustic monitoring

The pillow uses microphone input to observe sound intensity associated with possible snoring. The measurement is treated as a monitoring signal rather than a medical diagnosis.

Configurable threshold and rolling-window detection

The embedded detection design evaluates whether sufficient sound events occur inside a specified time window. The number of required events and the window duration can be configured, reducing reliance on a single isolated sound.

Air-pump and inflatable-bladder intervention

When the configured conditions are satisfied, the pillow can activate an air pump for a controlled duration. Inflation changes the pillow’s elevation to gently adjust the user’s head position.

Solenoid-valve release mechanism

A release valve allows the inflated bladder to return toward its resting state after an intervention. The application protocol also defines an open-valve command for supported firmware.

Bluetooth Low Energy communication

BLE provides energy-efficient communication between the pillow and the application. It supports device discovery, PIN-based pairing, event notifications, settings transmission, status handling, and supported safety commands.

Compact binary device protocol

The implemented application parser recognizes little-endian binary packets containing a protocol signature, event code, sound level, RMS value, pump duration, snore streak, and intervention flag. Invalid or undersized packets are rejected.

Remote settings and safety commands

The application encodes device settings into binary messages and defines commands for emergency pump stop and valve release. Values are normalized to established limits before transmission.

Mock hardware simulation

When native BLE is unavailable, especially in Expo Go and web-oriented development, the application uses a mock pillow service. It supplies reproducible sample events without pretending that a physical device is connected.

Implementation boundary: The mobile-side ESP32 protocol, BLE client, settings encoder, command encoder, and packet parser are present in this repository. The corresponding ESP32 firmware source is not included here; final hardware completion must therefore be verified against the separate firmware and physical prototype.

B. Mobile Application and Computational Algorithms

Cross-platform React Native application

React Native 0.76, React 18, Expo SDK 52, and TypeScript provide a shared codebase for Android and iOS. A web build is also supported, with safe limitations for native-only functions.

Profile, onboarding, and navigation workflow

The application restores or creates a profile and organizes Dashboard, Logs, Night, Assessment, Settings, Profile, and Wellness Plan History screens through stack and tab navigation.

Real and mock BLE service abstraction

react-native-ble-plx is used when an installed native build exposes the BLE module. The mock implementation is selected when the native module is unavailable, keeping the rest of the application operational during development.

Offline SQLite persistence

Expo SQLite stores user profiles, paired-device state, settings, sleep events, check-ins, saved wellness plans, notification preferences, and feedback. Write-ahead logging and timestamp indexes improve local reliability and chronological queries.

Storage migration and web compatibility

Migration routines preserve compatible values from earlier AsyncStorage and database schemas. A web-specific adapter provides browser-compatible persistence where native SQLite behavior is unavailable.

Daily statistical aggregation

Deterministic utilities calculate event count, average duration, intervention count, peak hour, and severity for a selected day or interval.

Night-session reconstruction

Events are grouped into an evening-to-morning window so that one sleep session is not incorrectly divided at midnight. The Night screen presents a chronological event and intervention timeline.

Monthly trend classification

Aggregated monthly measurements are compared to determine whether the observed pattern is improving, stable, or worsening. This provides longer-term context beyond a single night.

Intervention-outcome estimation

The analytics layer examines the timing of subsequent events after an intervention to estimate whether the adjustment appeared effective. This result is observational and is not presented as clinical proof.

Interactive visual analytics

react-native-chart-kit and react-native-svg render statistical cards, trend visualizations, hourly patterns, and historical summaries.

Deterministic wellness recommendation engine

Rule-based logic combines measured severity, monthly direction, check-in responses, and curated wellness knowledge. The same inputs produce predictable recommendations and explanations.

Optional on-device language-model inference

llama.rn and llama.cpp can run a downloaded model within the installed application. For non-critical assessments, the model personalizes low-risk wellness actions while allowing the user's check-in text to remain on the device.

Structured AI-output validation

Generated responses are constrained to an expected structure, parsed, checked, and rejected when invalid. Deterministic logic remains available if the model is unavailable or fails.

Critical-result generative-AI bypass

Critical or danger-level readings do not depend on model-generated advice. They retain conservative language encouraging professional evaluation so that an LLM cannot weaken the safety message.

Optional server-assisted assessment

The repository contains a server endpoint capable of using a local llama.cpp-compatible server or OpenAI. Provider credentials remain server-side and must never be exposed through an EXPO_PUBLIC_* variable.

Contextual daily check-in

Users may record activities and an optional note. The information supports reflection and personalization but is not used to claim causal relationships between an activity and snoring.

Wellness-plan history and feedback

Plans can be saved and reviewed. Users may mark actions as helpful, unhelpful, or not possible, enabling later recommendations to avoid unsuitable suggestions.

Local notification services

The application supports snore and intervention alerts, scheduled daily advice, model-download updates, and notifications when a personalized plan is ready.

Validation and resilience mechanisms

PINs, settings, BLE packets, database values, and AI output pass through validation. Error boundaries and platform-aware fallbacks reduce the effect of unsupported modules or malformed input.

Automated verification

Jest tests cover assessment behavior, statistics, wellness analytics, recommendation rules, LLM response parsing, knowledge selection, and PIN validation. TypeScript also checks the consistency of application interfaces.

C. Technology Stack and Operational Responsibilities

Layer or subsystem	Technologies	Operational responsibility
Smart-pillow controller	ESP32, microphone input, pump, valve, inflatable bladder	Acoustic monitoring, threshold evaluation, intervention control, and event reporting
Device communication	BLE GATT, react-native-ble-plx, compact binary packets	Pairing, event notification, configuration writes, and device commands
Mobile client	React Native 0.76, React 18, Expo SDK 52, TypeScript	Cross-platform user interface, onboarding, monitoring, assessment, history, and settings

Local persistence	Expo SQLite, AsyncStorage migration, browser storage adapter	Offline profiles, events, settings, check-ins, plans, preferences, and feedback
Analytics	TypeScript deterministic utilities	Daily/nightly/monthly aggregation, severity, peak periods, trends, and intervention estimates
Visualization	React Native Chart Kit, React Native SVG	Dashboard cards, charts, timelines, and trend presentation
On-device AI	llama.rn, llama.cpp, downloaded GGUF-compatible model	Optional local personalization for non-critical wellness guidance
Optional assessment server	Node.js, llama.cpp-compatible API or OpenAI provider	Server-assisted structured guidance when deliberately configured
Notifications	Expo Notifications	Snore/intervention alerts, daily advice, model status, and plan-ready alerts
Build and testing	EAS Build, Jest, React Test Renderer, TypeScript, ESLint	Native packaging, automated verification, static checking, and code-quality review

Current Completion Percentage

Provisional Overall Project Completion: 82%

This percentage is a repository-based engineering estimate, not a substitute for the group's approved capstone milestone rubric. The mobile software is substantially implemented and testable. Remaining risk is concentrated in physical-pillow verification, native production packaging, end-to-end field validation, and formal usability evaluation.

Subsystem or module	Estimated completion	Evidence and remaining work
Mobile interface and navigation	95%	Core screens and reusable components are present; final device-size and usability polishing remains.

Local persistence and migration	95%	SQLite tables, migration logic, indexed events, and web adapter are implemented; extended migration testing remains advisable.
Analytics and visualization	95%	Daily, nightly, monthly, peak-hour, severity, and intervention computations are implemented and tested.
Rule-based wellness guidance	95%	Safety-bounded recommendations, knowledge selection, feedback, and history are implemented.
Optional on-device AI	85%	Model download and llama.cpp integration exist; performance and memory validation on target devices remains.
BLE mobile integration	85%	Real client, mock fallback, parser, settings, and commands exist; full physical-device regression testing remains.
Notifications	85%	Multiple notification services are implemented; background behavior requires final native-device validation.
Smart-pillow hardware and firmware	65%	Hardware behavior and protocol are documented, but firmware source is not included in this repository; physical verification is required.
Automated testing and quality checks	85%	Type-check passes and 21 tests pass; linting is blocked by the missing Expo ESLint configuration package.
Android/iOS deployment	65%	EAS profiles are configured, but the recorded Android preview build ended in a

		Gradle error and requires repair and rebuild.
Field pilot and formal evaluation	30%	Final user acceptance, usability, reliability, and real-night evaluation evidence still needs to be collected.

Verification Performed on October 2, 2026

npm.cmd run type-check: Passed with no TypeScript errors.

npx.cmd jest --runInBand: Passed — 8 test suites and 21 tests.

npm.cmd run lint: Could not start because the ESLint configuration named expo is not installed or resolvable.

Latest recorded EAS Android preview build: Errored during the Gradle phase and produced no APK artifact.

Project Status

A. Features and Functions Already Working

User-profile creation, restoration, editing, and local persistence.

Seven-digit pairing-PIN input and format validation.

Automatic selection between real BLE and mock BLE implementations.

Native BLE discovery, connection, event subscription, and settings-write architecture.

Parsing of structured ESP32 pillow packets into typed sleep events.

Local storage of raw snoring and intervention events.

Device-setting storage for event threshold, pump duration, microphone shift, and rolling window.

Dashboard cards for event count, average duration, interventions, peak hour, and estimated effectiveness.

Daily, weekly, date-range, nightly, and monthly analytical views.

Historical event logs with filtering, sorting, and deletion behavior.

Night-session timeline and intervention-outcome presentation.

Daily activity check-in and optional free-text reflection.

Deterministic normal, elevated, and critical wellness guidance.

Safety-controlled critical results that bypass generative AI.

Optional local-model download and on-device inference integration.

Structured parsing and validation of LLM-generated recommendations.

Wellness-plan saving, history, user feedback, and regeneration support.

Local notifications for relevant pillow and assessment events.

Responsive application download screen.

Mock data suitable for repeatable demonstrations in Expo Go.

Automated tests for the principal analytical and recommendation utilities.

B. Features Currently Under Development or Requiring Final Validation

Repairing the Android EAS Gradle build and producing a verified installable APK.

Testing real BLE discovery, pairing, reconnection, notification streaming, settings writes, and safety commands against the final ESP32 firmware.

Confirming that firmware consumes the rolling-window setting and all defined command packets consistently.

Measuring pump timing, inflation height, valve-release behavior, noise, temperature, and repeated-cycle durability on the physical pillow.

Testing notification delivery when the app is backgrounded or the phone is locked.

Profiling on-device model download, initialization time, memory consumption, generation latency, and battery usage on target Android devices.

Installing the required Expo ESLint configuration and restoring a passing lint workflow.

Completing accessibility, small-screen, large-text, and error-recovery checks.

Conducting formal user acceptance testing and collecting approved software-quality and usability results.

C. Features Not Yet Implemented or Reserved for Future Development

Clinically validated sleep-apnea detection.

Oxygen-saturation, heart-rate, respiratory-rate, or polysomnography integration.

Multi-user household profiles connected to one pillow.

Optional encrypted cloud backup and cross-device synchronization.

Remote clinician portal or medical-record integration.

Federated or privacy-preserving population-level model training.

Predictive anomaly detection trained on a validated longitudinal dataset.

Over-the-air ESP32 firmware updating through the mobile application.

Data and Dataset Details

A. Current Data Sources

ESP32 event stream — Live pillow packets are designed to supply sound level, RMS intensity, event type, detection streak, intervention status, and pump duration.

Seeded prototype events — The mock BLE service provides approximately 90 days of deterministic sample events for repeatable interface and analytics demonstrations.

User profile data — Locally entered name and optional profile preferences support application personalization.

Daily check-in data — User-selected activities and an optional note provide contextual input for reflection and non-critical recommendations.

Device configuration data — Threshold, pump duration, microphone shift, and rolling-window values define pillow behavior.

Recommendation-feedback data — Helpful, unhelpful, or impractical ratings are stored for later plan refinement.

B. Types of Data

Time-series sleep-event data: Event identifier, timestamp, duration, severity, source, and raw packet information.

Acoustic summary data: ESP32-reported level and RMS measurements rather than continuous diagnostic audio recordings.

Intervention data: Whether inflation occurred and how long the pump operated.

Aggregated analytical data: Counts, averages, hourly distributions, nightly summaries, monthly trends, and estimated intervention outcomes.

Self-reported contextual data: Daily activities and optional notes supplied by the user.

Application-state data: Profile, paired device, settings, notification preferences, saved plans, and action feedback.

C. Data Scope and Limitations

Prototype demonstration data are generated locally and must not be described as clinical evidence.

The application's measurements are wellness-oriented observations and have not been validated against polysomnography.

Any final research report should state the actual number of physical test nights, users, pillow cycles, and collected events only after those values are documented.

Personally identifiable and wellness-related records remain local by default; optional server inference should be enabled only with appropriate consent and security controls.

Screenshot and Evidence Plan

Insert actual, current screenshots after each caption. Do not use mockups as proof of a completed physical function unless they are clearly labeled as simulations.

Figure 1. HAGOKILLER System Architecture

Show the microphone and ESP32, detection decision, air-pump intervention, BLE transfer, local database, analytics, rule-based guidance, and optional on-device model.

Figure 2. Smart-Pillow Hardware Prototype

Include a labeled photograph of the ESP32, microphone, relay or driver, pump, valve, bladder, power supply, and assembled pillow.

Figure 3. Device Pairing Screen

Show seven-digit PIN entry, validation feedback, and the successful connection state.

Figure 4. Dashboard and Snoring Analytics

Show event count, duration, interventions, peak period, severity, and trend charts.

Figure 5. Night Detail and Intervention Timeline

Show chronologically arranged snoring events, pump activations, and the observational effectiveness summary.

Figure 6. Historical Logs and Filters

Show stored event records, severity labels, duration, source, sorting, or date filtering.

Figure 7. Daily Assessment Check-In

Show activity choices and the optional user note used for contextual personalization.

Figure 8. Wellness Recommendation and Safety Boundary

Show a normal or elevated personalized plan and a separate critical state with deterministic professional-evaluation guidance.

Figure 9. Smart-Pillow Settings and Commands

Show threshold, rolling window, microphone shift, pump duration, connection information, emergency stop, and valve controls where available.

Figure 10. Automated Test Evidence

Include the terminal result showing 8 passing suites and 21 passing tests, together with the passing TypeScript check.

Figure 11. Native Build and Physical BLE Test Evidence

Once repaired, include the successful EAS APK build, installation on the test device, ESP32 connection, and a live event received from the pillow.

Immediate Next Milestones

Repair the missing ESLint Expo configuration and obtain a passing lint result.

Diagnose the recorded Gradle failure and produce a successful Android preview APK.

Validate every BLE packet and command against the final ESP32 firmware.

Execute a documented physical test matrix for sensing, intervention, valve release, reconnect behavior, and data synchronization.

Collect screenshots and photographs corresponding to Figures 1–11.

Conduct approved user acceptance and usability evaluation.

Replace provisional percentages with panel-approved milestone values and measured evidence.

Summary

HAGOKILLER has a substantially implemented mobile application with offline persistence, real and simulated BLE services, deterministic analytics, wellness guidance, notification support, and optional privacy-preserving on-device language-model inference. The strongest remaining work concerns physical smart-pillow verification, firmware-to-application integration, native Android packaging, code-quality configuration, and formal field evaluation. Completing these items will convert the current functional prototype into a defensibly validated capstone deliverable.